PYTHON ACTIVITY-BASED LEARNING
CASE STUDY 11

Examination Result and Grace-Mark Validation

B.Tech First Year – CSE | Python Programming

1. Problem Statement

Process marks across multiple subjects, apply subject-wise pass criteria and a specified grace-mark policy, and generate final classifications without exceeding maximum marks.

2. Objectives

- Calculate total and percentage from subject marks.
- Check subject-wise pass marks.
- Apply a limited grace-mark rule where permitted.
- Display final result and classification.

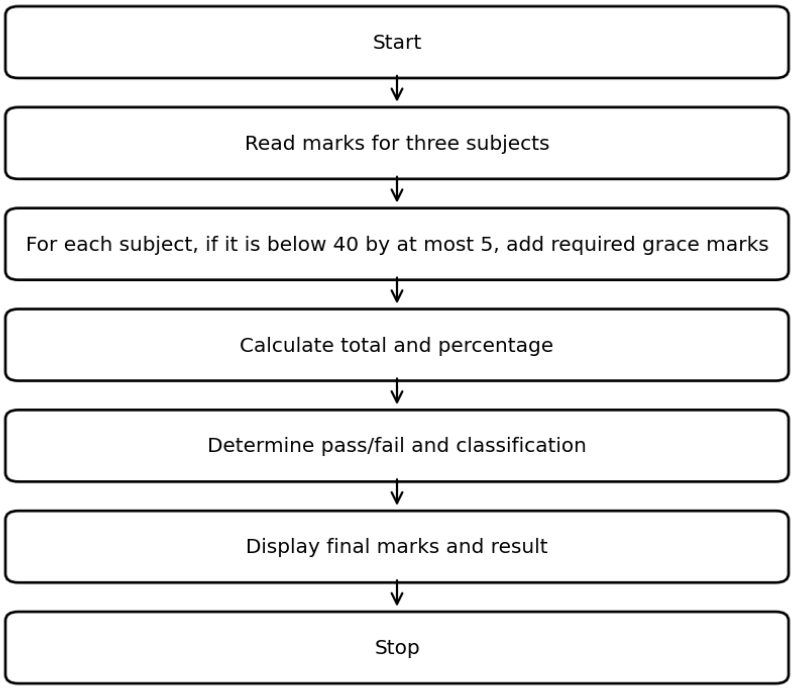
3. Assumptions

- There are 3 subjects, each out of 100.
- Pass mark is 40 in each subject.
- At most 5 grace marks may be added to one failed subject.
- Grace marks cannot make a subject exceed 100.
- Classification is Distinction at 75% or above, First Class at 60% or above, otherwise Pass.

4. Algorithm

1. Start
2. Read marks for three subjects
3. For each subject, if it is below 40 by at most 5, add required grace marks
4. Calculate total and percentage
5. Determine pass/fail and classification
6. Display final marks and result
7. Stop

5. Flowchart



6. Pseudocode

START

INPUT marks[0], marks[1], marks[2]

FOR each mark

  IF mark < 40 AND (40 - mark) <= 5 THEN mark = 40

END FOR

total = SUM(marks)

percentage = total / 3

IF any mark < 40: result = Fail

ELSE classify percentage

OUTPUT marks, total, percentage, result

STOP

7. Python Source Code

```
marks = [int(input("Enter marks for subject 1: ")),
        int(input("Enter marks for subject 2: ")),
        int(input("Enter marks for subject 3: "))]

pass_mark = 40
grace_limit = 5
grace_used = 0

for i in range(len(marks)):
```

```
    if marks[i] < pass_mark and pass_mark - marks[i] <= grace_limit:
        marks[i] = pass_mark
        grace_used += pass_mark - (marks[i] - (pass_mark - marks[i]))

total = sum(marks)
percentage = total / len(marks)

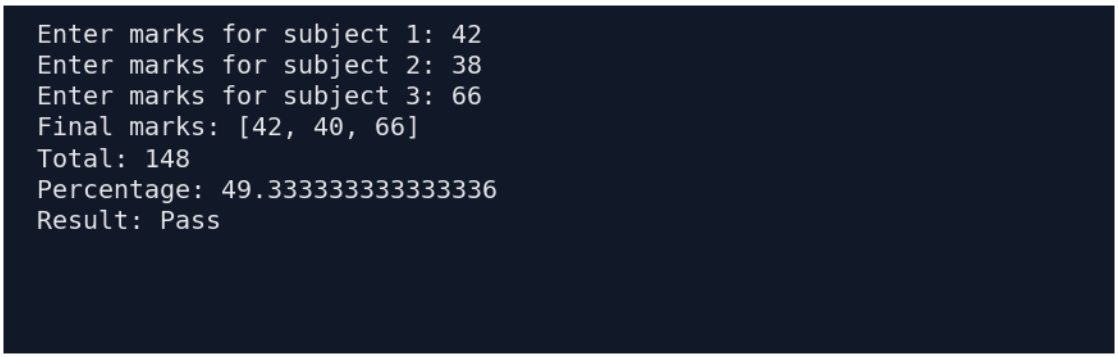
if min(marks) < pass_mark:
    result = "Fail"
elif percentage >= 75:
    result = "Distinction"
elif percentage >= 60:
    result = "First Class"
else:
    result = "Pass"

print("Final marks:", marks)
print("Total:", total)
print("Percentage:", percentage)
print("Result:", result)
```

8. Test Cases

Test	Input / Scenario	Expected Result
1	Subject 1: 42, Subject 2: 38, Subject 3: 66	Matches sample output below
2	Invalid / boundary values as applicable	Program handles conditions using stated rules

9. Output Screenshot



10. Sample Output

Enter marks for subject 1: 42
Enter marks for subject 2: 38
Enter marks for subject 3: 66
Final marks: [42, 40, 66]
Total: 148

Percentage: 49.333333333333336
Result: Pass

11. Conclusion

This case study demonstrates how basic Python concepts can be used to solve the problem of examination result and grace-mark validation. The program uses simple data structures, conditions, loops, and calculations to produce the required result.

Submitted by: _____ Roll No.: _____

PYTHON ACTIVITY-BASED LEARNING
CASE STUDY 17

Student Performance Trend Analysis

B.Tech First Year – CSE | Python Programming

1. Problem Statement

Compare assessment scores across multiple examinations, calculate improvements and declines, identify consistently weak subjects, and classify students according to defined criteria.

2. Objectives

- Store scores from two exams.
- Calculate subject-wise changes.
- Identify improvement or decline.
- Find the lowest-scoring subject.

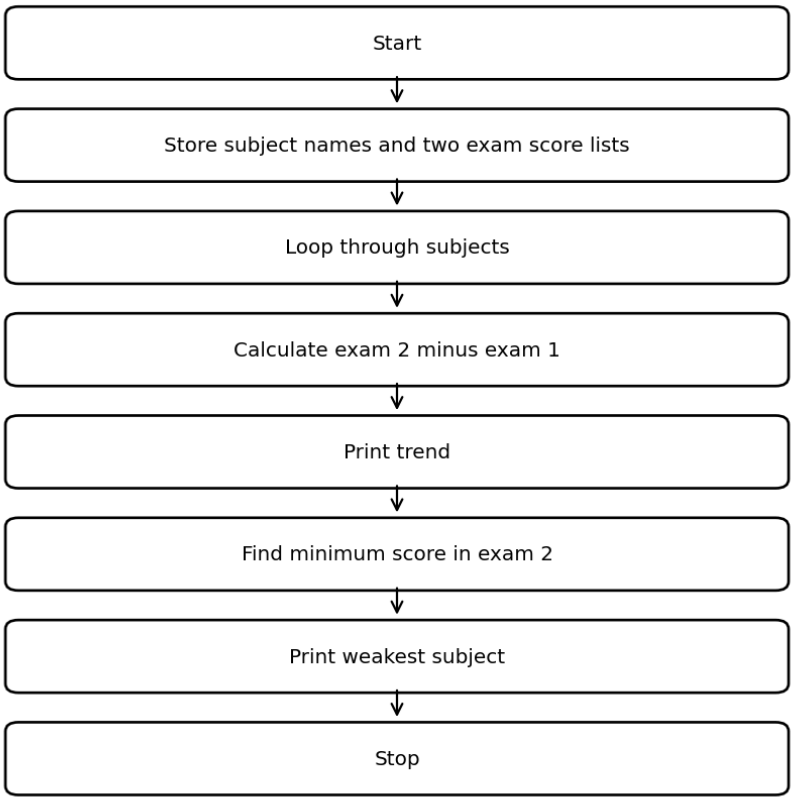
3. Assumptions

- Three subjects are considered.
- Each subject has marks out of 100.
- A change of 5 marks or more is reported as a notable trend.

4. Algorithm

1. Start
2. Store subject names and two exam score lists
3. Loop through subjects
4. Calculate exam 2 minus exam 1
5. Print trend
6. Find minimum score in exam 2
7. Print weakest subject
8. Stop

5. Flowchart



6. Pseudocode

START

subjects = [Python, Maths, English]

exam1 = [65,72,58]

exam2 = [78,68,64]

FOR i from 0 to 2

  change = exam2[i] - exam1[i]

  DISPLAY subject and change trend

END FOR

weakest = subject at minimum exam2 score

DISPLAY weakest

STOP

7. Python Source Code

```
subjects = ["Python", "Maths", "English"]
exam1 = [65, 72, 58]
exam2 = [78, 68, 64]
```

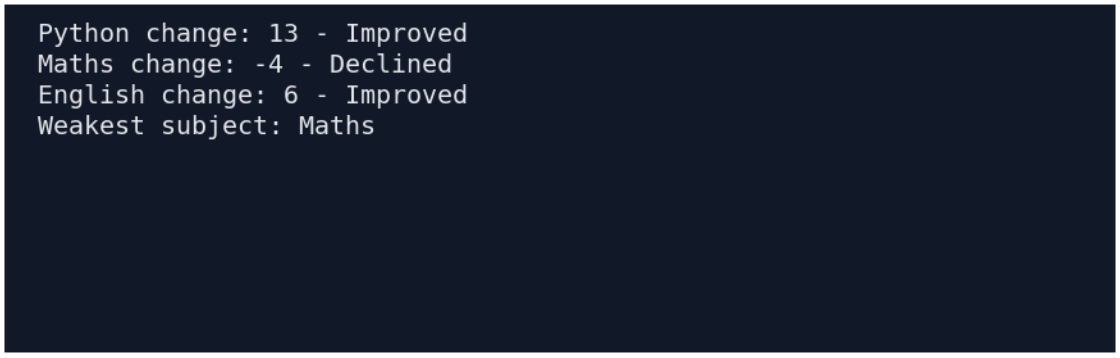
```
for i in range(len(subjects)):
    change = exam2[i] - exam1[i]
    if change > 0:
        trend = "Improved"
    elif change < 0:
        trend = "Declined"
    else:
        trend = "No change"
    print(subjects[i], "change:", change, "-", trend)

weak_index = exam2.index(min(exam2))
print("Weakest subject:", subjects[weak_index])
```

8. Test Cases

Test	Input / Scenario	Expected Result
1	Use the sample values defined in the program	Matches sample output below
2	Invalid / boundary values as applicable	Program handles conditions using stated rules

9. Output Screenshot



10. Sample Output

Python change: 13 - Improved
Maths change: -4 - Declined
English change: 6 - Improved
Weakest subject: Maths

11. Conclusion

This case study demonstrates how basic Python concepts can be used to solve the problem of student performance trend analysis. The program uses simple data structures, conditions, loops, and calculations to produce the required result.

Submitted by: _____ Roll No.: _____

PYTHON ACTIVITY-BASED LEARNING
CASE STUDY 19

Multi-Product Shopping Budget Constraint

B.Tech First Year – CSE | Python Programming

1. Problem Statement

Evaluate a shopping basket containing multiple products and quantities, apply discount conditions, and determine which purchases must be reduced to remain within a fixed budget.

2. Objectives

- Calculate product-wise costs.
- Apply a discount when the subtotal reaches a threshold.
- Compare the final bill with a budget.
- Display the amount over or under budget.

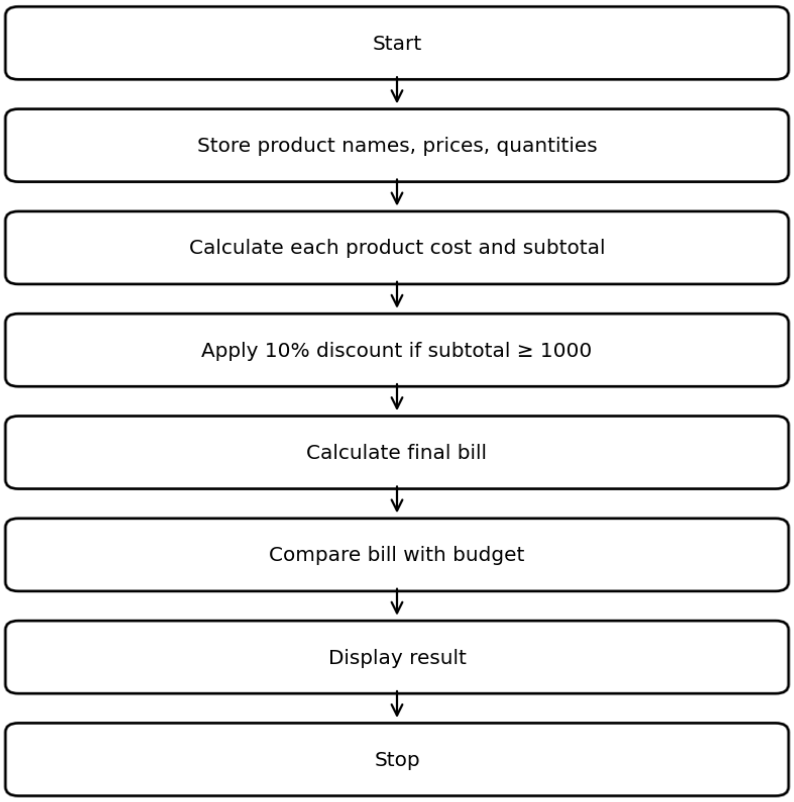
3. Assumptions

- Products are stored in lists.
- A 10% discount applies when subtotal is at least ₹1,000.
- Budget is ₹1,500.
- No tax is included.

4. Algorithm

1. Start
2. Store product names, prices, quantities
3. Calculate each product cost and subtotal
4. Apply 10% discount if subtotal ≥ 1000
5. Calculate final bill
6. Compare bill with budget
7. Display result
8. Stop

5. Flowchart



6. Pseudocode

START

subtotal = 0

FOR each product

  cost = price * quantity

  subtotal = subtotal + cost

END FOR

IF subtotal >= 1000: discount = subtotal * 0.10

ELSE discount = 0

total = subtotal - discount

IF total > budget: DISPLAY over budget

ELSE DISPLAY remaining balance

STOP

7. Python Source Code

```
products = ["Notebook", "Pen", "Bag"]
prices = [80, 20, 900]
quantities = [3, 5, 1]
```

```
subtotal = 0
for i in range(len(products)):
    cost = prices[i] * quantities[i]
    subtotal += cost
    print(products[i], ":", cost)

discount = 0
if subtotal >= 1000:
    discount = subtotal * 0.10

total = subtotal - discount
budget = 1500

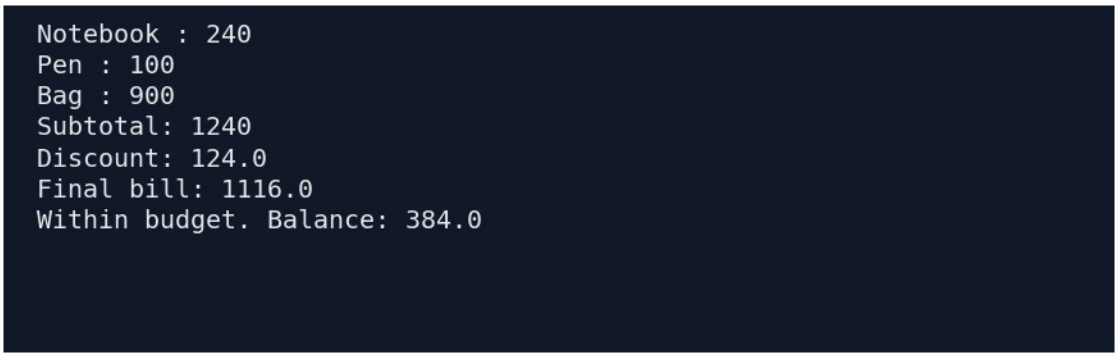
print("Subtotal:", subtotal)
print("Discount:", discount)
print("Final bill:", total)

if total > budget:
    print("Over budget by:", total - budget)
else:
    print("Within budget. Balance:", budget - total)
```

8. Test Cases

Test	Input / Scenario	Expected Result
1	Use the sample values defined in the program	Matches sample output below
2	Invalid / boundary values as applicable	Program handles conditions using stated rules

9. Output Screenshot



10. Sample Output

Notebook : 240
Pen : 100
Bag : 900
Subtotal: 1240

Discount: 124.0
Final bill: 1116.0
Within budget. Balance: 384.0

11. Conclusion

This case study demonstrates how basic Python concepts can be used to solve the problem of multi-product shopping budget constraint. The program uses simple data structures, conditions, loops, and calculations to produce the required result.

Submitted by: _____ Roll No.: _____

PYTHON ACTIVITY-BASED LEARNING
CASE STUDY 20

University Admission Decision Engine

B.Tech First Year – CSE | Python Programming

1. Problem Statement

Evaluate candidates using subject eligibility, aggregate marks, course-specific thresholds, and tie-breaking criteria; generate a ranked list of eligible applicants.

2. Objectives

- Check eligibility conditions.
- Calculate aggregate percentage.
- Select eligible candidates.
- Rank candidates by score.

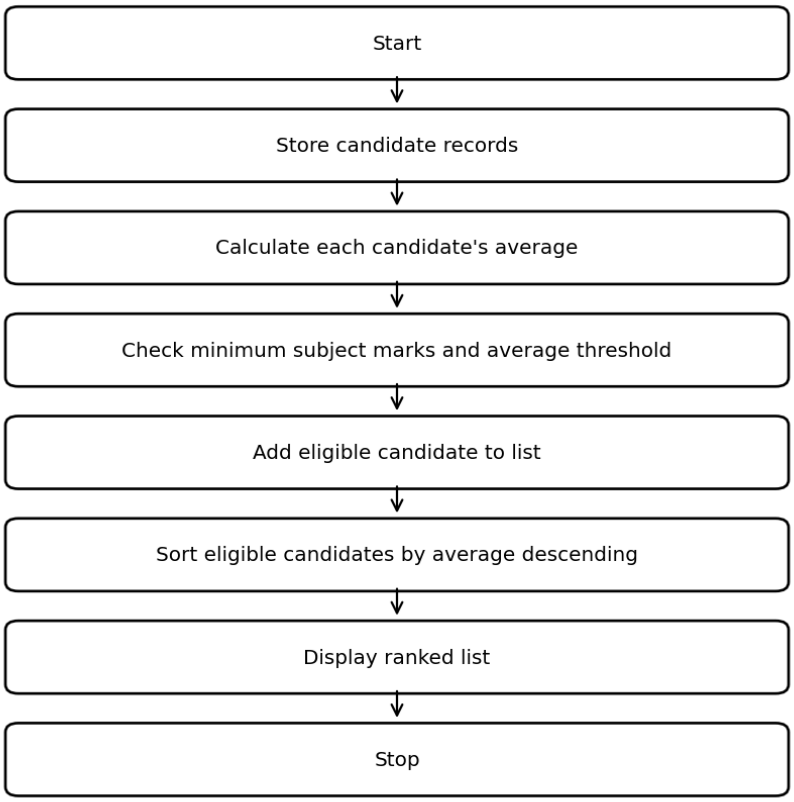
3. Assumptions

- Each candidate has Math, Physics, and Computer marks.
- Minimum mark in each subject is 40.
- Aggregate percentage must be at least 60.
- Candidates are ranked by aggregate percentage, highest first.

4. Algorithm

1. Start
2. Store candidate records
3. Calculate each candidate's average
4. Check minimum subject marks and average threshold
5. Add eligible candidate to list
6. Sort eligible candidates by average descending
7. Display ranked list
8. Stop

5. Flowchart



6. Pseudocode

START

eligible = empty list

FOR each candidate tuple

  average = sum(subject marks) / 3

  IF every subject >= 40 AND average >= 60

    ADD (name, average) to eligible

  END IF

END FOR

SORT eligible by average descending

DISPLAY ranking

STOP

7. Python Source Code

```
candidates = [
    ("Asha", 80, 75, 90),
    ("Ravi", 55, 62, 70),
    ("Mina", 35, 88, 91),
    ("Kiran", 70, 70, 70)
]
```

```
eligible = []
for name, math, physics, computer in candidates:
    average = (math + physics + computer) / 3
    if math >= 40 and physics >= 40 and computer >= 40 and average >= 60:
        eligible.append((name, average))

eligible.sort(key=lambda item: item[1], reverse=True)

print("Admission ranking")
for rank, candidate in enumerate(eligible, start=1):
    print(rank, candidate[0], round(candidate[1], 2))
```

8. Test Cases

Test	Input / Scenario	Expected Result
1	Use the sample values defined in the program	Matches sample output below
2	Invalid / boundary values as applicable	Program handles conditions using stated rules

9. Output Screenshot



10. Sample Output

Admission ranking
1 Asha 81.67
2 Kiran 70.0
3 Ravi 62.33

11. Conclusion

This case study demonstrates how basic Python concepts can be used to solve the problem of university admission decision engine. The program uses simple data structures, conditions, loops, and calculations to produce the required result.

Submitted by: _____ Roll No.: _____

PYTHON ACTIVITY-BASED LEARNING
CASE STUDY 21

Traffic Violation Penalty Calculator

B.Tech First Year – CSE | Python Programming

1. Problem Statement

Process multiple violations using vehicle category, offence type, repeat-offence status, and predefined fine rules; calculate the final penalty and identify disqualification conditions.

2. Objectives

- Store violation types and fines.
- Apply repeat-offence penalties.
- Calculate total fine.
- Identify disqualification when a severe offence occurs.

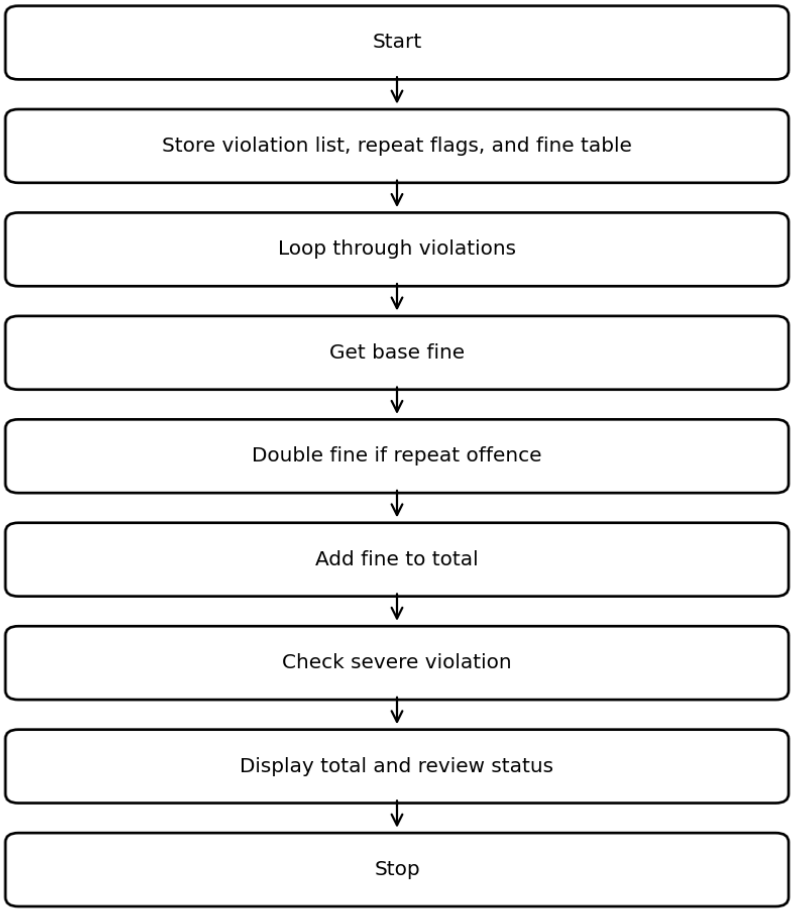
3. Assumptions

- Fine amounts are sample values for this programming exercise only.
- Helmet violation = ₹500; signal violation = ₹1,000; overspeeding = ₹2,000.
- Repeat offences double the fine.
- A severe overspeeding offence is marked for review.

4. Algorithm

1. Start
2. Store violation list, repeat flags, and fine table
3. Loop through violations
4. Get base fine
5. Double fine if repeat offence
6. Add fine to total
7. Check severe violation
8. Display total and review status
9. Stop

5. Flowchart



6. Pseudocode

```
START
total = 0
FOR each violation
  fine = fine_table[violation]
  IF repeat: fine = fine * 2
  total = total + fine
  IF violation is overspeed: review = True
END FOR
DISPLAY total
IF review: DISPLAY review message
STOP
```

7. Python Source Code

```
violations = ["helmet", "signal", "overspeed"]
repeat = [False, True, False]
fine_table = {"helmet": 500, "signal": 1000, "overspeed": 2000}

total = 0
review = False

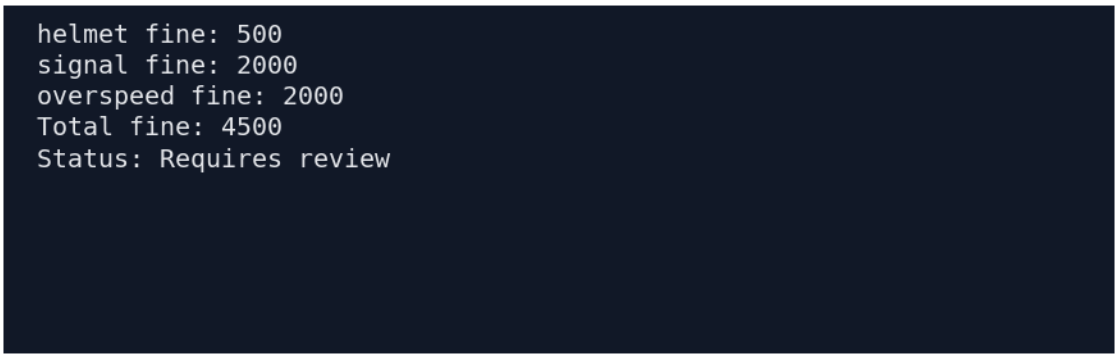
for i in range(len(violations)):
    fine = fine_table[violations[i]]
    if repeat[i]:
        fine = fine * 2
    total += fine
    print(violations[i], "fine:", fine)
    if violations[i] == "overspeed":
        review = True

print("Total fine:", total)
if review:
    print("Status: Requires review")
```

8. Test Cases

Test	Input / Scenario	Expected Result
1	Use the sample values defined in the program	Matches sample output below
2	Invalid / boundary values as applicable	Program handles conditions using stated rules

9. Output Screenshot



10. Sample Output

helmet fine: 500
signal fine: 2000
overspeed fine: 2000
Total fine: 4500

Status: Requires review

11. Conclusion

This case study demonstrates how basic Python concepts can be used to solve the problem of traffic violation penalty calculator. The program uses simple data structures, conditions, loops, and calculations to produce the required result.

Submitted by: _____ Roll No.: _____

PYTHON ACTIVITY-BASED LEARNING
CASE STUDY 26

Nested Loop Examination Seating Generator

B.Tech First Year – CSE | Python Programming

1. Problem Statement

Generate a seating arrangement with row and column identifiers, reserved positions, alternate seating restrictions, and fixed classroom capacities.

2. Objectives

- Generate seats using nested loops.
- Mark reserved positions.
- Apply alternate seating restrictions.
- Display the seating layout.

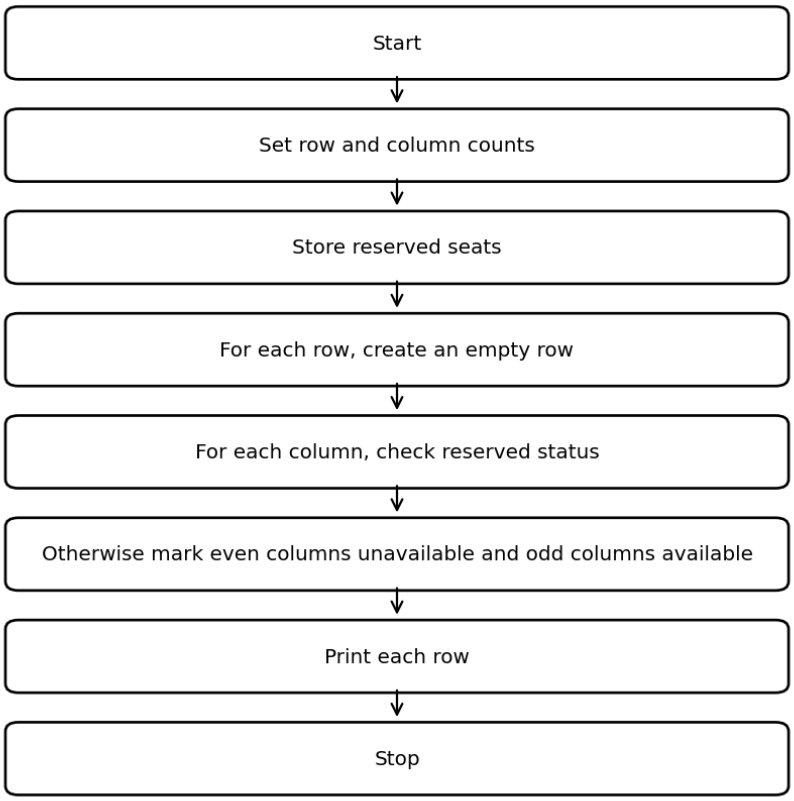
3. Assumptions

- Classroom has 3 rows and 4 columns.
- Seats (1,2) and (2,3) are reserved.
- Only odd-numbered columns are available for this exercise.

4. Algorithm

1. Start
2. Set row and column counts
3. Store reserved seats
4. For each row, create an empty row
5. For each column, check reserved status
6. Otherwise mark even columns unavailable and odd columns available
7. Print each row
8. Stop

5. Flowchart



6. Pseudocode

```
START
rows = 3; columns = 4
reserved = [(1,2),(2,3)]
FOR r = 1 to rows
  FOR c = 1 to columns
    IF (r,c) reserved: seat = R
    ELSE IF c even: seat = X
    ELSE: seat = S
    DISPLAY seat
  END FOR
END FOR
STOP
```

7. Python Source Code

```
rows = 3
columns = 4
reserved = [(1, 2), (2, 3)]
```

```
for r in range(1, rows + 1):
    line = []
    for c in range(1, columns + 1):
        if (r, c) in reserved:
            seat = "R"
        elif c % 2 == 0:
            seat = "X"
        else:
            seat = "S"
        line.append(seat)
    print("Row", r, ":", " ".join(line))
```

8. Test Cases

Test	Input / Scenario	Expected Result
1	Use the sample values defined in the program	Matches sample output below
2	Invalid / boundary values as applicable	Program handles conditions using stated rules

9. Output Screenshot



10. Sample Output

Row 1 : S R S X
Row 2 : S X S R
Row 3 : S X S X

11. Conclusion

This case study demonstrates how basic Python concepts can be used to solve the problem of nested loop examination seating generator. The program uses simple data structures, conditions, loops, and calculations to produce the required result.

Submitted by: _____ Roll No.: _____

PYTHON ACTIVITY-BASED LEARNING
CASE STUDY 27

Advanced Numeric Pattern Generator

B.Tech First Year – CSE | Python Programming

1. Problem Statement

Produce a hollow diamond or symmetric numerical pattern based on user-specified dimensions, with validation for invalid dimensions and carefully aligned output.

2. Objectives

- Read a pattern size.
- Validate that size is positive.
- Generate a symmetric number pattern using loops.
- Display the pattern.

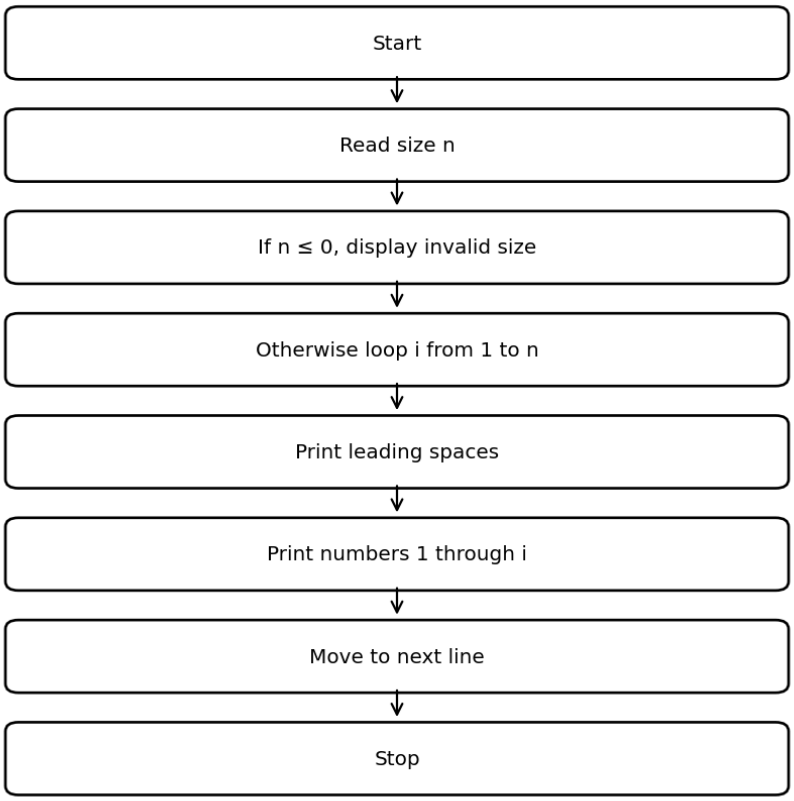
3. Assumptions

- The example uses size 4.
- The pattern is a centered number pyramid.
- Each row prints numbers from 1 through its row number.

4. Algorithm

1. Start
2. Read size n
3. If n ≤ 0, display invalid size
4. Otherwise loop i from 1 to n
5. Print leading spaces
6. Print numbers 1 through i
7. Move to next line
8. Stop

5. Flowchart



6. Pseudocode

```
START
INPUT n
IF n <= 0
  DISPLAY Invalid size
ELSE
  FOR i = 1 to n
    PRINT n-i spaces
    FOR j = 1 to i: PRINT j
    PRINT newline
  END FOR
END IF
STOP
```

7. Python Source Code

```
n = int(input("Enter pattern size: "))

if n <= 0:
```

```
    print("Invalid size")
else:
    for i in range(1, n + 1):
        print(" " * (n - i), end="")
        for j in range(1, i + 1):
            print(j, end=" ")
        print()
```

8. Test Cases

Test	Input / Scenario	Expected Result
1	Pattern size: 4	Matches sample output below
2	Invalid / boundary values as applicable	Program handles conditions using stated rules

9. Output Screenshot



10. Sample Output

```
Enter pattern size: 4
   1
  1 2
 1 2 3
1 2 3 4
```

11. Conclusion

This case study demonstrates how basic Python concepts can be used to solve the problem of advanced numeric pattern generator. The program uses simple data structures, conditions, loops, and calculations to produce the required result.

Submitted by: _____ Roll No.: _____

PYTHON ACTIVITY-BASED LEARNING
CASE STUDY 31

String-Based Examination Answer Evaluation

B.Tech First Year – CSE | Python Programming

1. Problem Statement

Compare answer strings against an answer key, calculate correct, incorrect, and unanswered responses, apply negative marking, and produce student-wise results.

2. Objectives

- Compare student answers with an answer key.
- Count correct, wrong, and blank answers.
- Apply negative marking.
- Calculate final score.

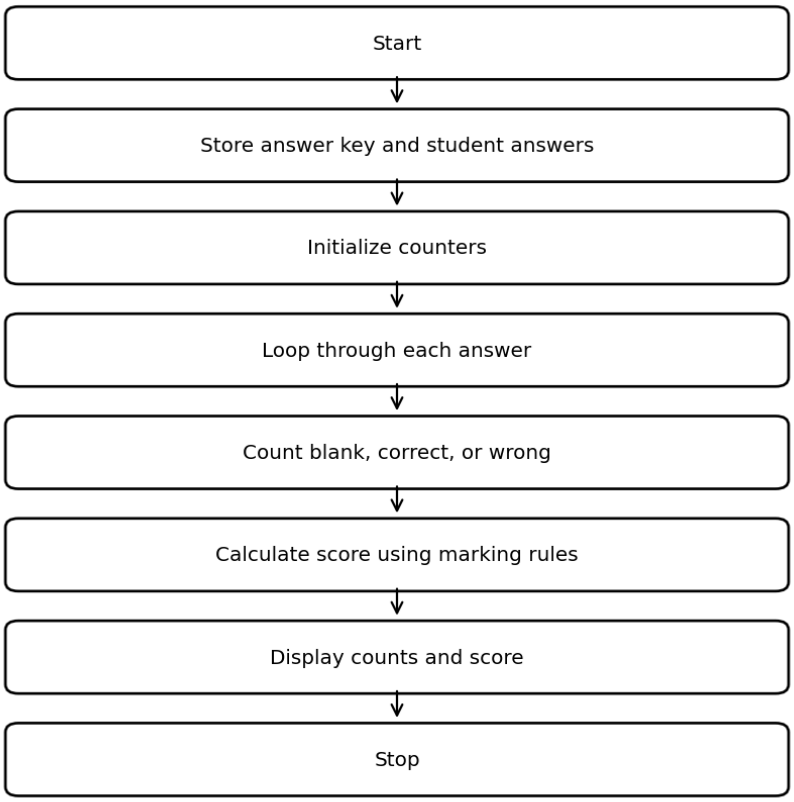
3. Assumptions

- Each correct answer gives 2 marks.
- Each wrong answer deducts 0.5 marks.
- Blank answers give 0 marks.
- Answers are case-insensitive.

4. Algorithm

1. Start
2. Store answer key and student answers
3. Initialize counters
4. Loop through each answer
5. Count blank, correct, or wrong
6. Calculate score using marking rules
7. Display counts and score
8. Stop

5. Flowchart



6. Pseudocode

START

key = answer key

answers = student answers

correct = wrong = blank = 0

FOR each answer index

  IF answer blank: blank += 1

  ELSE IF uppercase answer equals key: correct += 1

  ELSE: wrong += 1

END FOR

score = correct*2 - wrong*0.5

DISPLAY counts and score

STOP

7. Python Source Code

```
key = ["A", "C", "B", "D", "A"]
answers = ["a", "C", "D", "", "A"]

correct = 0
```

```
wrong = 0
blank = 0

for i in range(len(key)):
    answer = answers[i].upper()
    if answer == "":
        blank += 1
    elif answer == key[i]:
        correct += 1
    else:
        wrong += 1

score = correct * 2 - wrong * 0.5

print("Correct:", correct)
print("Wrong:", wrong)
print("Unanswered:", blank)
print("Final score:", score)
```

8. Test Cases

Test	Input / Scenario	Expected Result
1	Use the sample values defined in the program	Matches sample output below
2	Invalid / boundary values as applicable	Program handles conditions using stated rules

9. Output Screenshot



10. Sample Output

Correct: 3
Wrong: 1
Unanswered: 1
Final score: 5.5

11. Conclusion

This case study demonstrates how basic Python concepts can be used to solve the problem of string-based examination answer evaluation. The program uses simple data structures, conditions, loops, and calculations to produce the required result.

Submitted by: _____ Roll No.: _____

PYTHON ACTIVITY-BASED LEARNING
CASE STUDY 47

Sports Tournament Ranking System

B.Tech First Year – CSE | Python Programming

1. Problem Statement

Store match results and team scores, calculate total points and tie-break values, and produce rankings under a specified competition policy.

2. Objectives

- Store team match points.
- Calculate total points.
- Use score difference as tie-break.
- Display ranked teams.

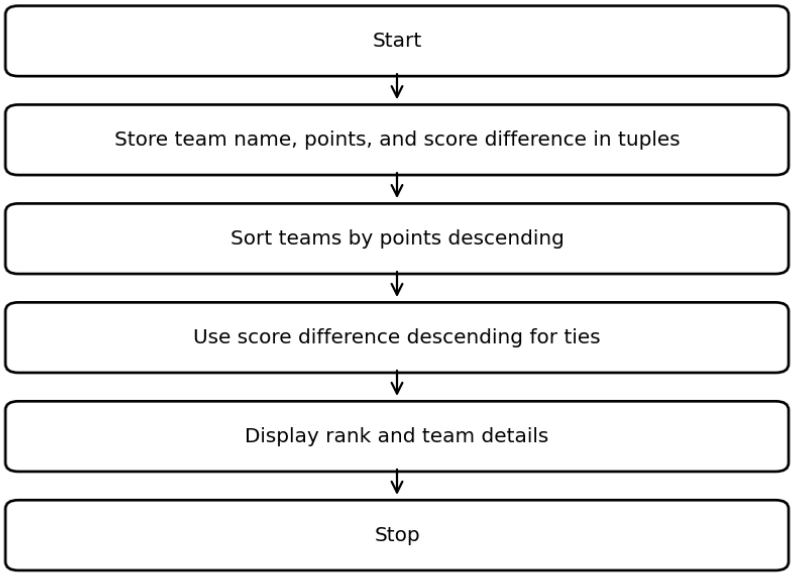
3. Assumptions

- Win gives 3 points, draw gives 1, loss gives 0.
- For this simplified example, supplied total points and score difference are used.
- Ranking is by points, then score difference.

4. Algorithm

1. Start
2. Store team name, points, and score difference in tuples
3. Sort teams by points descending
4. Use score difference descending for ties
5. Display rank and team details
6. Stop

5. Flowchart



6. Pseudocode

START

teams = list of (name, points, difference) tuples

SORT by points descending, then difference descending

rank = 1

FOR each team

  DISPLAY rank, name, points, difference

  rank = rank + 1

END FOR

STOP

7. Python Source Code

```
teams = [
    ("Falcons", 7, 8),
    ("Tigers", 7, 12),
    ("Wolves", 5, 4),
    ("Eagles", 9, 6)
]

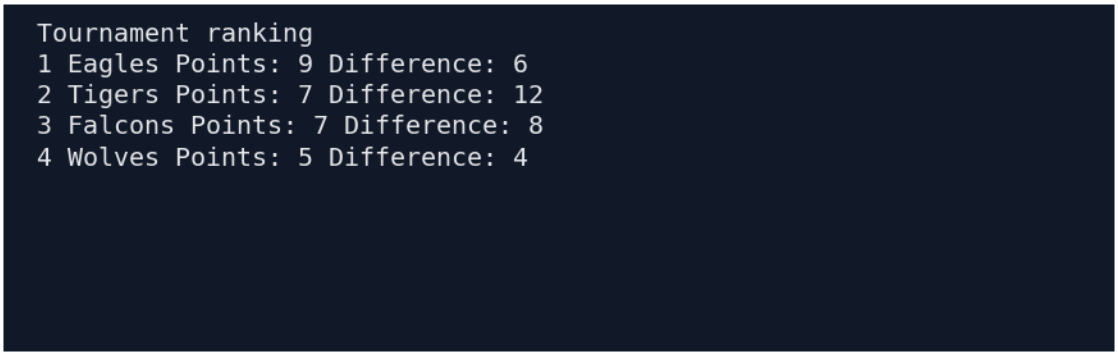
teams.sort(key=lambda team: (team[1], team[2]), reverse=True)

print("Tournament ranking")
for rank, (name, points, difference) in enumerate(teams, start=1):
    print(rank, name, "Points:", points, "Difference:", difference)
```

8. Test Cases

Test	Input / Scenario	Expected Result
1	Use the sample values defined in the program	Matches sample output below
2	Invalid / boundary values as applicable	Program handles conditions using stated rules

9. Output Screenshot



10. Sample Output

Tournament ranking
1 Eagles Points: 9 Difference: 6
2 Tigers Points: 7 Difference: 12
3 Falcons Points: 7 Difference: 8
4 Wolves Points: 5 Difference: 4

11. Conclusion

This case study demonstrates how basic Python concepts can be used to solve the problem of sports tournament ranking system. The program uses simple data structures, conditions, loops, and calculations to produce the required result.

Submitted by: _____ Roll No.: _____

PYTHON ACTIVITY-BASED LEARNING
CASE STUDY 52

Hackathon Team Formation and Ranking

B.Tech First Year – CSE | Python Programming

1. Problem Statement

Analyse student scores and team compositions, validate team sizes, aggregate round-wise scores, and resolve tied rankings using predefined rules.

2. Objectives

- Check team size.
- Add scores from multiple rounds.
- Rank valid teams.
- Display invalid team-size warnings.

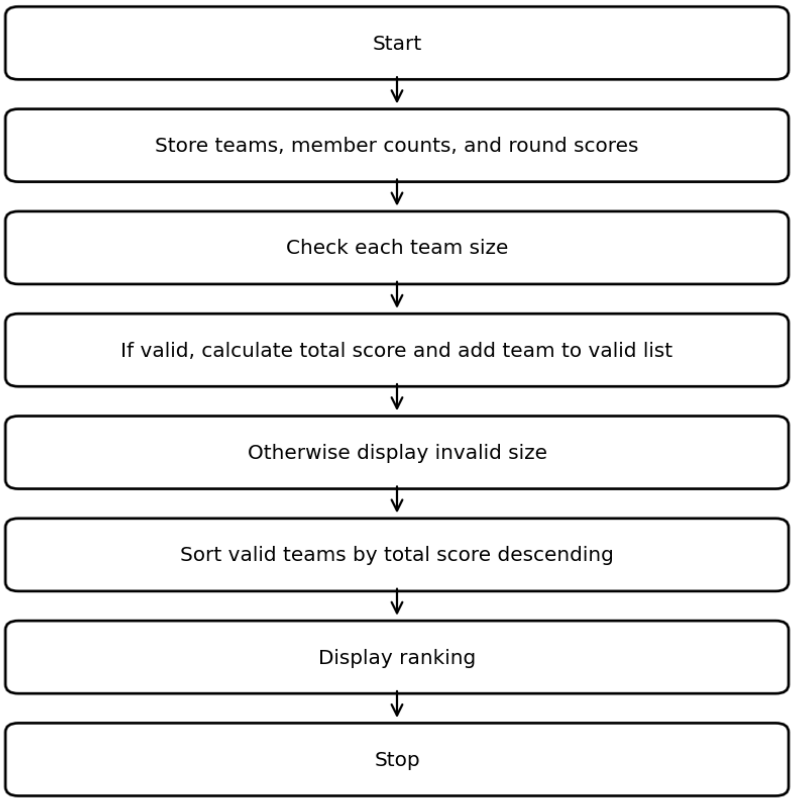
3. Assumptions

- Each team must have 3 to 5 members.
- Two round scores are provided.
- Teams are ranked by total score, highest first.
- This example contains team names and member counts, not personal data.

4. Algorithm

1. Start
2. Store teams, member counts, and round scores
3. Check each team size
4. If valid, calculate total score and add team to valid list
5. Otherwise display invalid size
6. Sort valid teams by total score descending
7. Display ranking
8. Stop

5. Flowchart



6. Pseudocode

START

valid = empty list

FOR each team

IF members >= 3 AND members <= 5

total = SUM(round scores)

ADD (name,total) to valid

ELSE DISPLAY invalid team size

END FOR

SORT valid by total descending

DISPLAY ranked teams

STOP

7. Python Source Code

```
teams = [
    ("CodeStars", 4, [80, 90]),
    ("ByteBuilders", 3, [85, 85]),
    ("LogicCrew", 2, [95, 90]),
    ("Debuggers", 5, [70, 88])
]
```

```
valid = []
for name, members, scores in teams:
    if 3 <= members <= 5:
        total = sum(scores)
        valid.append((name, total))
    else:
        print(name, "invalid team size:", members)

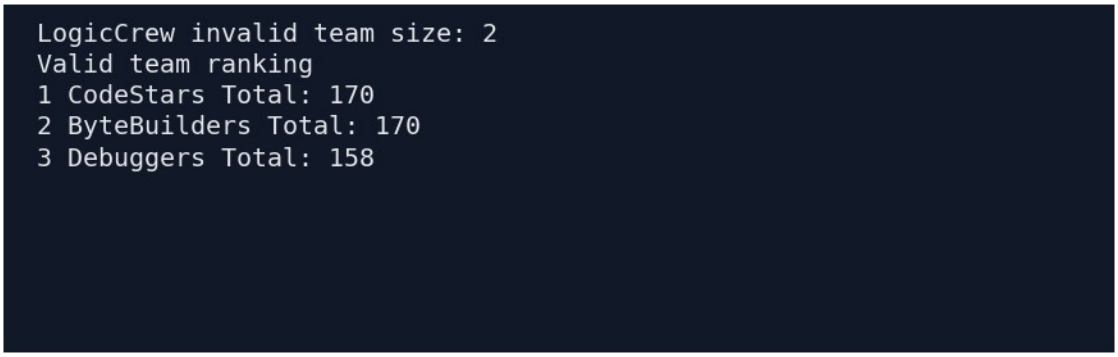
valid.sort(key=lambda team: team[1], reverse=True)

print("Valid team ranking")
for rank, (name, total) in enumerate(valid, start=1):
    print(rank, name, "Total:", total)
```

8. Test Cases

Test	Input / Scenario	Expected Result
1	Use the sample values defined in the program	Matches sample output below
2	Invalid / boundary values as applicable	Program handles conditions using stated rules

9. Output Screenshot



10. Sample Output

LogicCrew invalid team size: 2
Valid team ranking
1 CodeStars Total: 170
2 ByteBuilders Total: 170
3 Debuggers Total: 158

11. Conclusion

This case study demonstrates how basic Python concepts can be used to solve the problem of hackathon team formation and ranking. The program uses simple data structures, conditions, loops, and calculations to produce the required result.

Submitted by: _____ Roll No.: _____